

Hathor - Weekly Report - Week 2

Summary

This week focused on establishing the technical foundation, architecture standards, and fullstack execution roadmap for the Hathor platform. Primary things that have been done this week include finalizing the Execution Plan, initializing the pnpm monorepo and Docker Compose topology, embedding contract validation in CI, approving system documentation across docs/, and kicking off implementation across Gateway, Auth, Web Shell, and Catalog microservices (Issues 1.3 through 1.6).

Completed Tasks

Full-Stack Execution Plan Finalized

- **Overview & Strategy:** Designed and finalized the comprehensive Full-Stack Execution Plan outlining the delivery roadmap for the Hathor platform. The plan establishes strict governance across planning, contracts, backend services, frontend integration, testing, security, and operations.
- **Board Configuration & Workflow Rules:** Structured a Project Delivery Board with 12 tracking fields (Status, Priority, Area, Milestone, Owner, Reviewer, Dates, Dependencies, Environment, Release Blocker) and strict lifecycle stages (Backlog, Ready, In Progress, In Review, QA, Blocked, Done).
- **Definitions of Ready & Done:** Established clear criteria requiring mapped contracts, assigned owners, explicit acceptance criteria, zero raw secret logging, Drizzle service migrations, and passing automated test suites before issues can close.
- **Team Responsibility Allocation:** Accountable ownership assigned across 4 primary teammates covering Platform/Gateway (Teammate 1), Auth/Commerce (Teammate 2), Catalog/Library (Teammate 3), and Web Frontend/Generated Client (Teammate 4).
- **Milestone Dependency Mapping:** Defined the complete 7-phase delivery path (M0 Architecture & Contracts -> M1 Platform & Gateway -> M2 Catalog & Cart -> M3 Payments & Entitlements -> M4 Downloads & Creator AI -> M5 Integration & Recovery -> M6 Release Hardening).

Monorepo & Local Compose Topology Initialized

- **Status:** Completed
- **Monorepo Architecture:** Initialized pnpm workspace root containing apps/api-gateway, four isolated microservices (apps/auth-service, apps/catalog-service, apps/commerce-service, apps/library-service), and apps/web frontend shell.
- **Shared Transport Boundary:** Created packages/contracts strictly for auto-generated DTOs and HTTP transport schemas, preventing cross-domain database model coupling.
- **Multi-Container Composition:** Docker Compose topology orchestrating 4 independent PostgreSQL database instances (one per domain service), RabbitMQ message broker, Redis key-value cache, Memcached, API Gateway proxy, and Web shell.
- **Port Isolation & Security Perimeter:** Only API Gateway and Web Shell bind host network ports (3000/8080). Downstream microservices and databases are isolated strictly on private container subnets.

Contract Validation & Test Foundation Established

- **Status:** Completed

- **Schema Validation Tooling:** Integrated OpenAPI (public-api.openapi.yaml, internal-api.openapi.yaml) and AsyncAPI (domain-events.asyncapi.yaml) linting and validation tasks into workspace scripts and GitHub Actions CI workflows.
- **Test Compose Profile:** Configured an isolated container profile for automated testing, enabling ephemeral database startup and pre-seeded fixtures.
- **Distributed Tracing Interceptor:** Built global X-Correlation-ID tracking middleware across HTTP request headers and RabbitMQ message headers.

Technical Documentation Completed

- **Location:** Hathor-docs
- **Architecture & Decision Records:** Finalized system topology specifications and ADR-001 (August Local-First Release Architecture) in docs/architecture/.
- **Contract Specifications:** Approved public and internal OpenAPI specifications alongside AsyncAPI event definitions in docs/contracts/.
- **Security & Reliability Runbooks:** Established authentication controls (Argon2id, RS256 JWT, Turnstile validation), transactional outbox patterns, DLQ replay runbooks, and delivery guidelines in docs/security/, docs/reliability/, and docs/delivery/.

Work In Progress (Started This Week)

1.3: Gateway Edge Controls

- **Status:** In Progress
- **Namespace Proxying:** Routing public endpoints (/api/v1/user/* to Auth Service; /api/v1/store/* to Catalog Service; /api/v1/cart/* to Commerce Service; /api/v1/inventory/* to Library Service).
- **Private Path Isolation:** Enforcing edge rules that block external inbound requests from reaching internal endpoints (/internal/*).
- **Edge Middleware:** Implementing strict origin CORS, Helmet Content Security Policy (CSP, X-Frame-Options: DENY), request payload size caps, IP rate limiting, and standard error normalization ({ "error": { "code", "message", "correlationId" } }).

1.4: Auth, Sessions, & Service Tokens

- **Status:** In Progress
- **User Registration & Password Security:** Argon2id password hashing and Cloudflare Turnstile CAPTCHA server-side validation behind a test adapter.
- **RS256 Access JWTs & JWKS:** Issuing 10-15 minute RS256-signed Access JWTs with public key distribution at /api/v1/user/.well-known/jwks.json.
- **Rotating Refresh Token Families:** Refresh tokens issued in HttpOnly, Secure, SameSite=Lax cookies scoped to /api/v1/user/refresh, with automatic family revocation upon reuse detection.
- **Authorization Versioning & Service Identity:** authorization_version tracking in database for session invalidation on password reset or role changes, secret-gated CLI admin bootstrap command, and 5-minute internal service-to-service JWT issuance.

1.5: Web Shell & Auth Experience

- **Status:** In Progress
- **TanStack Application Shell:** Building apps/web frontend shell using TanStack Start, Router, and Query, integrated with the auto-generated TypeScript API client from packages/contracts.
- **In-Memory Access Token Architecture:** Access JWTs stored strictly in JavaScript React Context state (never in localStorage/sessionStorage) to protect against XSS token theft.
- **Session Recovery & Route Guards:** Automatic background session refresh on app boot via HttpOnly cookies, origin/CSRF protection, and TanStack Router beforeLoad guards for protected routes (/library, /profile).

1.6: Seed Catalog Read Path & Storefront Shell

- **Status:** In Progress
- **Database Seeding:** PostgreSQL migrations in catalog-service populating demo published games, tag taxonomies, creator records, and precise prices in EGP currency.
- **Public Read APIs:** Exposing `/api/v1/store/games` (catalog browse with tag filtering and pagination) and `/api/v1/store/games/:id` (game detail view), filtering out draft or suspended items.
- **Web Storefront UI:** Frontend browse and game detail components in `apps/web` rendering seeded catalog data using TanStack Query and generated API types.

Next Week Objectives

1. Complete implementation and test coverage for Issues 1.3, 1.4, 1.5, and 1.6.
2. Verify local end-to-end integration user flow: user registration -> login -> GET `/user/me` -> browse catalog storefront.
3. Begin Milestone 2: Catalog game status transitions, Store Page Designer draft editing, cart management, and payment-pending order creation.